

Exercise 1: Inventory Management System

1. Aim

To develop an Inventory Management System using Java and implement efficient product storage and retrieval using a `HashMap`. The system supports adding, updating, deleting, and displaying products.

2. Problem Understanding

An inventory management system stores information about products in a warehouse. As the number of products increases, efficient storage and retrieval become important. Data Structures and Algorithms help improve the performance of operations such as searching, updating, and deleting products.

For this problem, a **HashMap** is chosen because each product has a unique Product ID, which serves as the key. A `HashMap` provides fast insertion, search, update, and deletion operations with an average time complexity of **O(1)**, making it suitable for handling large inventories.

3. Suitable Data Structure

Chosen Data Structure: `HashMap<Integer, Product>`

Reason:

- Product ID is unique.
- Direct access using the Product ID.
- Faster than an `ArrayList` for searching, updating, and deleting products.
- Efficient for large datasets.

4. Algorithm

1. Create a `Product` class with the attributes:
 - Product ID
 - Product Name
 - Quantity
 - Price
2. Create an `Inventory` class.

3. Store products in a HashMap using Product ID as the key.
4. Implement methods to:
 - Add a product.
 - Update a product.
 - Delete a product.
 - Display all products.
5. Test all operations in the main() method.

5.Program/Output:

```
week1 > Algorithm > J Exercise1.java > Exercise1 > main(String[])  
1  import java.util.HashMap;  
2  
3  class Product{  
4      private int productId;  
5      private String productName;  
6      private int quality;  
7      private double price;  
8  
9      Product(int productId,String productName, int quality,double price){  
10         this.productId = productId;  
11         this.productName = productName;  
12         this.quality = quality;  
13         this.price= price;  
14     }  
15     public int getProductId(){  
16         return productId;  
17     }  
18     public void setQuality(int quality){  
19         this.quality = quality;  
20     }  
21     public void setPrice(double price){  
22         this.price = price;  
23     }  
24     @Override  
25     public String toString(){  
26         return "Product ID: "+productId+  
27             ", Name: "+productName+", Quality: "+quality  
28             +", Price: Rs."+price;  
29     }  
30 }  
31 class Inventory{  
32     private HashMap<Integer,Product> products = new HashMap<>();  
33     public void addProduct(Product product){  
34         products.put(product.getProductId(),product);  
35         System.out.println(x: "Product Added successfully.");  
36     }  
37     public void updateProduct(int id,int quantity, double price){  
38         Product product = products.get(id);  
39         if(product!=null){  
40             product.setPrice(price);  
41             product.setQuality(quantity);  
42             System.out.println(x: "Product Updated Successfully.");  
43         }  
44         else{  
45             System.out.println(x: "Product Not found");  
46         }  
47     }  
48     public void deleteProduct(int id){  
49         if(products.remove(id)!=null){  
50             System.out.println(x: "Product Deleted Successfully");  
51         }  
52         else{  
53             System.out.println(x: "Product Not Found.");  
54         }  
55     }  
56     public void displayProducts(){  
57         System.out.println(x: "\nCurrent Inventory:");  
58         for(Product product:products.values()){  
59             System.out.println(product);  
60         }  
61     }  
62 }
```

```

63 public class Exercise1{
64     public static void main(String args[])
65     {
66         Inventory inventory = new Inventory();
67
68         inventory.addProduct(new Product(productId: 101, productName: "Laptop", quality: 10, price: 55000));
69         inventory.addProduct(new Product(productId: 103, productName: "Mouse", quality: 15, price: 670));
70         inventory.addProduct(new Product(productId: 102, productName: "Keyboard", quality: 30, price: 1200));
71
72         inventory.displayProducts();
73
74         System.out.println();
75
76         inventory.updateProduct(id: 101, quantity: 15, price: 53200);
77
78         inventory.displayProducts();
79
80         System.out.println();
81
82         inventory.deleteProduct(id: 102);
83
84         inventory.displayProducts();
85     }
86 }

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```

c2280d97420f71afa9ba416523e0f2f\redhat.java\jdt_ws\week_7a7b8c98\bin' 'Exercise1'
Product Added successfully.
Product Added successfully.
Product Added successfully.

Current Inventory:
Product ID: 101, Name: Laptop, Quality: 10, Price: Rs.55000.0
Product ID: 102, Name: Keyboard, Quality: 30, Price: Rs.1200.0
Product ID: 103, Name: Mouse, Quality: 15, Price: Rs.670.0

Product Updated Successfully.

Current Inventory:
Product ID: 101, Name: Laptop, Quality: 15, Price: Rs.53200.0
Product ID: 102, Name: Keyboard, Quality: 30, Price: Rs.1200.0
Product ID: 103, Name: Mouse, Quality: 15, Price: Rs.670.0

Product Deleted Successfully

Current Inventory:
Product ID: 101, Name: Laptop, Quality: 15, Price: Rs.53200.0
Product ID: 103, Name: Mouse, Quality: 15, Price: Rs.670.0
PS C:\Users\Krishnavishwa\OneDrive\Documents\Material\cognizant\week>
    
```

6. Time Complexity:

Operation	HashMap Time Complexity
Add Product	O(1)
Search Product	O(1)
Update Product	O(1)
Delete Product	O(1)

7. Result

Thus, the **Inventory Management System** was successfully implemented using Java. A HashMap was used to efficiently store and manage product records. The system supports adding, updating, deleting, and displaying products with an average time complexity of **O(1)** for the major operations, making it suitable for large-scale inventory management.